

```

// ===== FINIX OS v7.7 - STABLE =====
// Creator: Nurudeen Al Haitami (Alkha)
// Architecture: ARM64 (simulated)
// Features: Multi-user, Firewall, Network Tools, Package Manager, Notes, Weather
//
=====
==

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <unistd.h>
#include <dirent.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <signal.h>
#include <setjmp.h>
#include <errno.h>
#include <pwd.h>
#include <grp.h>
#include <ctype.h>

// ===== KONFIGURASI =====
#define MAX_NAME 64
#define MAX_PATH 256
#define MAX_LINE 512
#define MAX_USERS 32
#define MAX_GROUPS 16
#define MAX_HISTORY 100

// ===== WARNA ANSI =====
#define COLOR_RED "\033[31m"
#define COLOR_GREEN "\033[32m"
#define COLOR_YELLOW "\033[33m"
#define COLOR_BLUE "\033[34m"
#define COLOR_MAGENTA "\033[35m"
#define COLOR_CYAN "\033[36m"
#define COLOR_RESET "\033[0m"

// ===== BAHASA =====
typedef enum { LANG_EN, LANG_ID } Language;
Language current_lang = LANG_EN;

// ===== REFERENCE COUNTING =====
typedef int refcount_t;

```

```

static inline void refcount_set(refcount_t *ref, int val) {
    *ref = val;
}

static inline void refcount_inc(refcount_t *ref) {
    (*ref)++;
}

static inline void refcount_dec(refcount_t *ref) {
    (*ref)--;
}

// ===== ATOMIC OPERATIONS (Simulated)
// =====
static inline int _sync_lock_test_and_set(int *lock, int val) {
    int old = *lock;
    *lock = val;
    return old;
}

// ===== LINKED LIST =====
struct list_head {
    struct list_head *next, *prev;
};

#define LIST_HEAD_INIT(name) { &(name), &(name) }
#define LIST_HEAD(name) struct list_head name = LIST_HEAD_INIT(name)

static inline void INIT_LIST_HEAD(struct list_head *list) {
    list->next = list;
    list->prev = list;
}

static inline void _list_add(struct list_head *new, struct list_head *prev, struct list_head *next)
{
    next->prev = new;
    new->next = next;
    new->prev = prev;
    prev->next = new;
}

static inline void list_add(struct list_head *new, struct list_head *head) {
    _list_add(new, head, head->next);
}

static inline void list_add_tail(struct list_head *new, struct list_head *head) {
    _list_add(new, head->prev, head);
}

```

```

static inline void _list_del(struct list_head *prev, struct list_head *next) {
    prev->next = next;
    next->prev = prev;
}

static inline void list_del(struct list_head *entry) {
    _list_del(entry->prev, entry->next);
    entry->next = NULL;
    entry->prev = NULL;
}

#define list_entry(ptr, type, member) \
    ((type *)((char *)(ptr) - (size_t)&((type *)0)->member)))

#define list_for_each(pos, head) \
    for (pos = (head)->next; pos != (head); pos = pos->next)

#define list_for_each_safe(pos, n, head) \
    for (pos = (head)->next, n = pos->next; pos != (head); pos = n, n = pos->next)

// ===== MEMORY CACHE =====
typedef struct kmem_cache {
    char name[32];
    size_t obj_size;
    int obj_count;
    int max_objs;
    void **objects;
    int *free_list;
    int free_count;
    int lock;
} kmem_cache_t;

kmem_cache_t *user_cache, *group_cache, *firewall_rule_cache;

kmem_cache_t *kmem_cache_create(const char *name, size_t obj_size, int max_objs) {
    kmem_cache_t *cache = malloc(sizeof(kmem_cache_t));
    if (!cache) return NULL;
    strncpy(cache->name, name, 31);
    cache->name[31] = '\0';
    cache->obj_size = obj_size;
    cache->obj_count = 0;
    cache->max_objs = max_objs;
    cache->objects = malloc(sizeof(void *) * max_objs);
    cache->free_list = malloc(sizeof(int) * max_objs);
    cache->free_count = max_objs;
    cache->lock = 0;
    for (int i = 0; i < max_objs; i++) {

```

```

        cache->free_list[i] = i;
        cache->objects[i] = NULL;
    }
    return cache;
}

void *kmem_cache_alloc(kmem_cache_t *cache) {
    while (_sync_lock_test_and_set(&cache->lock, 1)) {}
    if (cache->free_count == 0) {
        cache->lock = 0;
        return NULL;
    }
    int idx = cache->free_list[--cache->free_count];
    if (!cache->objects[idx])
        cache->objects[idx] = malloc(cache->obj_size);
    if (!cache->objects[idx]) {
        cache->lock = 0;
        return NULL;
    }
    memset(cache->objects[idx], 0, cache->obj_size);
    cache->obj_count++;
    cache->lock = 0;
    return cache->objects[idx];
}

void kmem_cache_free(kmem_cache_t *cache, void *obj) {
    while (_sync_lock_test_and_set(&cache->lock, 1)) {}
    for (int i = 0; i < cache->max_objs; i++) {
        if (cache->objects[i] == obj) {
            cache->free_list[cache->free_count++] = i;
            cache->obj_count--;
            break;
        }
    }
    cache->lock = 0;
}

void kmem_cache_destroy(kmem_cache_t *cache) {
    for (int i = 0; i < cache->max_objs; i++)
        if (cache->objects[i]) free(cache->objects[i]);
    free(cache->objects);
    free(cache->free_list);
    free(cache);
}

void *kmemdup(const void *src, size_t len) {
    void *p = malloc(len);
    if (p) memcpy(p, src, len);
}

```

```

    return p;
}

char *kstrdup(const char *s) {
    if (!s) return NULL;
    size_t len = strlen(s) + 1;
    char *p = malloc(len);
    if (p) memcpy(p, s, len);
    return p;
}

// ===== USER & GROUP =====
typedef struct finix_group {
    int gid;
    char name[MAX_NAME];
    int permissions;
    refcount_t refcnt;
    struct list_head list;
} finix_group_t;

typedef struct finix_user {
    int uid;
    char username[MAX_NAME];
    char password_hash[MAX_NAME];
    int gid;
    int *groups;
    int ngroups;
    refcount_t refcnt;
    struct list_head list;
} finix_user_t;

struct list_head user_list, group_list;
int next_uid = 1000, next_gid = 100;
finix_user_t *current_user = NULL;

int group_search(int *groups, int ngroups, int gid) {
    for (int i = 0; i < ngroups; i++)
        if (groups[i] == gid) return 1;
    return 0;
}

void simple_hash(const char *input, char *output) {
    unsigned long hash = 5381;
    int c;
    while ((c = *input++))
        hash = ((hash << 5) + hash) + c;
    sprintf(output, "%lx", hash);
}

```

```

void init_user_system(void) {
    INIT_LIST_HEAD(&user_list);
    INIT_LIST_HEAD(&group_list);

    finix_group_t *g;
    g = kmem_cache_alloc(group_cache);
    g->gid = next_gid++;
    strcpy(g->name, "wheel");
    g->permissions = 2;
    refcount_set(&g->refcnt, 1);
    list_add_tail(&g->list, &group_list);

    g = kmem_cache_alloc(group_cache);
    g->gid = next_gid++;
    strcpy(g->name, "users");
    g->permissions = 1;
    refcount_set(&g->refcnt, 1);
    list_add_tail(&g->list, &group_list);

    g = kmem_cache_alloc(group_cache);
    g->gid = next_gid++;
    strcpy(g->name, "guests");
    g->permissions = 0;
    refcount_set(&g->refcnt, 1);
    list_add_tail(&g->list, &group_list);

    finix_user_t *u = kmem_cache_alloc(user_cache);
    u->uid = 0;
    strcpy(u->username, "root");
    simple_hash("finix123", u->password_hash);
    u->gid = 100;
    u->ngroups = 1;
    u->groups = malloc(sizeof(int));
    u->groups[0] = 100;
    refcount_set(&u->refcnt, 1);
    list_add_tail(&u->list, &user_list);
    current_user = u;
}

int authenticate_user(const char *username, const char *password) {
    char hash[MAX_NAME];
    simple_hash(password, hash);
    struct list_head *pos;
    finix_user_t *u;
    list_for_each(pos, &user_list) {
        u = list_entry(pos, finix_user_t, list);
        if (strcmp(u->username, username) == 0 &&

```

```

        strcmp(u->password_hash, hash) == 0) {
            current_user = u;
            return 1;
        }
    }
    return 0;
}

int user_has_permission(int required_perm) {
    if (!current_user) return 0;
    if (current_user->uid == 0) return 1;
    struct list_head *pos;
    finix_group_t *g;
    list_for_each(pos, &group_list) {
        g = list_entry(pos, finix_group_t, list);
        if (group_search(current_user->groups, current_user->ngroups, g->gid) &&
            g->permissions >= required_perm)
            return 1;
    }
    return 0;
}

// ===== COMMAND HISTORY =====
char *history[MAX_HISTORY];
int history_count = 0;

void add_to_history(const char *cmd) {
    if (history_count < MAX_HISTORY) {
        history[history_count++] = kstrdup(cmd);
    } else {
        free(history[0]);
        for (int i = 1; i < MAX_HISTORY; i++)
            history[i - 1] = history[i];
        history[MAX_HISTORY - 1] = kstrdup(cmd);
    }
}

void show_history(void) {
    if (current_lang == LANG_ID)
        printf(COLOR_CYAN "\n📖 Riwayat Perintah:\n" COLOR_RESET);
    else
        printf(COLOR_CYAN "\n📖 Command History:\n" COLOR_RESET);
    for (int i = 0; i < history_count; i++)
        printf("%3d: %s\n", i + 1, history[i]);
}

void cmd_history_search(char *keyword) {
    if (!keyword || strlen(keyword) == 0) {

```

```

    show_history();
    return;
}
if (current_lang == LANG_ID)
    printf(COLOR_CYAN "\n📖 Riwayat mengandung '%s':\n" COLOR_RESET, keyword);
else
    printf(COLOR_CYAN "\n📖 History containing '%s':\n" COLOR_RESET, keyword);
int found = 0;
for (int i = 0; i < history_count; i++) {
    if (strstr(history[i], keyword)) {
        printf("%3d: %s\n", i + 1, history[i]);
        found++;
    }
}
if (!found) {
    if (current_lang == LANG_ID)
        printf("Tidak ada riwayat yang mengandung '%s'\n", keyword);
    else
        printf("No history containing '%s'\n", keyword);
}
}

// ===== ERROR RECOVERY =====
jmp_buf crash_env;
int crash_counter = 0;
time_t last_crash_time = 0;

void crash_handler(int sig) {
    time_t now = time(NULL);
    if (now - last_crash_time > 60) {
        crash_counter = 0;
        last_crash_time = now;
    }
    if (crash_counter++ < 5) {
        printf(COLOR_RED "\n⚠️ FinixOS crash detected! Restarting...\n" COLOR_RESET);
        longjmp(crash_env, 1);
    } else if (crash_counter == 5) {
        printf(COLOR_RED "\n❌ Too many crashes! Entering safe mode...\n"
COLOR_RESET);
        longjmp(crash_env, 2);
    }
}

void setup_error_recovery(void) {
    signal(SIGSEGV, crash_handler);
    signal(SIGABRT, crash_handler);
    signal(SIGFPE, crash_handler);
}

```



```
// ===== BOOT SCREEN =====

void boot_screen(void) {
    system("clear");
    printf(COLOR_CYAN);
    printf(" _____ \n");
    printf("| ____(_ )  (_ )  / _ \ \ / _ \ \n");
    printf("| |__ _ _ _ _ _ _ _ || || | \n");
    printf("| _ | || ' \ \ | \ \ \ / | | | | \n");
    printf("| | | | | | | | > < | | _ | | _ \n");
    printf("| _ | | _ | | _ | / _ \ \ \ \ \ / \ \ \ \ \n");
    printf(COLOR_RESET);
    printf(COLOR_GREEN "FinixOS v7.7 - Stable Edition\n" COLOR_RESET);
    printf("Creator: Nurudeen Al Haitami (Alkha)\n\n");
    printf("Booting: [");
    fflush(stdout);
    for (int i = 0; i <= 20; i++) {
        printf("#");
        fflush(stdout);
        usleep(50000);
    }
    printf("] 100%%\n");
    printf(COLOR_GREEN "[OK]" COLOR_RESET " Kernel loaded\n");
    printf(COLOR_GREEN "[OK]" COLOR_RESET " Memory management initialized\n");
    printf(COLOR_GREEN "[OK]" COLOR_RESET " User system initialized\n");
    printf(COLOR_GREEN "[OK]" COLOR_RESET " Security modules active\n");
    printf(COLOR_GREEN "[OK]" COLOR_RESET " Network tools ready\n");
    printf(COLOR_GREEN "[OK]" COLOR_RESET " Firewall active\n");
    printf(COLOR_GREEN "[OK]" COLOR_RESET " Package manager ready\n");
    printf(COLOR_GREEN "[OK]" COLOR_RESET " Languages: C, Python, Node.js\n\n");
}

// ===== FIREWALL =====

typedef struct firewall_rule {
    char proto[8];
    int port;
    char action[8];
    int packets_blocked;
    struct list_head list;
} firewall_rule_t;

struct list_head firewall_rules;

void firewall_init(void) {
    INIT_LIST_HEAD(&firewall_rules);
}

void cmd_firewall_add(char *proto, int port, char *action) {
```

```

if (!user_has_permission(1)) {
    if (current_lang == LANG_ID)
        printf(COLOR_RED "❌ Izin ditolak!\n" COLOR_RESET);
    else
        printf(COLOR_RED "❌ Permission denied!\n" COLOR_RESET);
    return;
}
firewall_rule_t *r = kmem_cache_alloc(firewall_rule_cache);
strcpy(r->proto, proto);
r->port = port;
strcpy(r->action, action);
r->packets_blocked = 0;
list_add_tail(&r->list, &firewall_rules);
if (current_lang == LANG_ID)
    printf(COLOR_GREEN "✅ Aturan ditambahkan: %s port %d %s\n" COLOR_RESET,
proto, port, action);
else
    printf(COLOR_GREEN "✅ Rule added: %s port %d %s\n" COLOR_RESET, proto,
port, action);
}

```

```

void cmd_firewall_list(void) {
    if (current_lang == LANG_ID)
        printf(COLOR_CYAN "\n📋 Aturan Firewall:\n" COLOR_RESET);
    else
        printf(COLOR_CYAN "\n📋 Firewall Rules:\n" COLOR_RESET);
    printf("Proto Port Action Blocked\n");
    printf("-----\n");
    struct list_head *pos;
    firewall_rule_t *r;
    list_for_each(pos, &firewall_rules) {
        r = list_entry(pos, firewall_rule_t, list);
        printf("%-6s %-5d %-8s %d\n", r->proto, r->port, r->action, r->packets_blocked);
    }
}

```

```

void cmd_firewall_delete(int port) {
    struct list_head *pos, *n;
    firewall_rule_t *r;
    int found = 0;
    list_for_each_safe(pos, n, &firewall_rules) {
        r = list_entry(pos, firewall_rule_t, list);
        if (r->port == port) {
            list_del(&r->list);
            kmem_cache_free(firewall_rule_cache, r);
            found = 1;
            break;
        }
    }
}

```

```

}
if (found) {
    if (current_lang == LANG_ID)
        printf(COLOR_GREEN "✅ Aturan port %d dihapus\n" COLOR_RESET, port);
    else
        printf(COLOR_GREEN "✅ Rule for port %d deleted\n" COLOR_RESET, port);
} else {
    if (current_lang == LANG_ID)
        printf(COLOR_RED "❌ Aturan port %d tidak ditemukan\n" COLOR_RESET, port);
    else
        printf(COLOR_RED "❌ Rule for port %d not found\n" COLOR_RESET, port);
}
}
}

```

// ===== NETWORK TOOLS =====

```

void cmd_netstat(void) {
    if (current_lang == LANG_ID)
        printf(COLOR_CYAN "\n🌐 Koneksi Jaringan:\n" COLOR_RESET);
    else
        printf(COLOR_CYAN "\n🌐 Network Connections:\n" COLOR_RESET);
    if (system("which netstat > /dev/null 2>&1") == 0) {
        system("netstat -tuln | head -20");
        return;
    }
    FILE *f = fopen("/proc/net/tcp", "r");
    if (!f) {
        if (current_lang == LANG_ID)
            printf("Informasi jaringan tidak tersedia\n");
        else
            printf("No network info available\n");
        return;
    }
    printf("Proto Local Address      Foreign Address    State\n");
    printf("-----\n");
    char line[256];
    fgets(line, sizeof(line), f);
    while (fgets(line, sizeof(line), f)) {
        char local_addr[65], rem_addr[65];
        int lport, rport, state_val;
        sscanf(line, "%*d: %64[0-9A-F]:%X %64[0-9A-F]:%X %X",
            local_addr, &lport, rem_addr, &rport, &state_val);
        unsigned int lip[4], rip[4];
        sscanf(local_addr, "%2x%2x%2x%2x", &lip[3], &lip[2], &lip[1], &lip[0]);
        sscanf(rem_addr, "%2x%2x%2x%2x", &rip[3], &rip[2], &rip[1], &rip[0]);
        printf("TCP  %d.%d.%d.%d:%d", lip[0], lip[1], lip[2], lip[3], lport);
        printf(" %d.%d.%d.%d:%d", rip[0], rip[1], rip[2], rip[3], rport);
        switch (state_val) {
            case 1: printf(" ESTABLISHED\n"); break;

```

```

        case 6: printf(" TIME_WAIT\n"); break;
        case 10: printf(" LISTEN\n"); break;
        default: printf(" UNKNOWN\n");
    }
}
fclose(f);
}

void cmd_nmap(char *host) {
    if (!host || strlen(host) == 0) {
        if (current_lang == LANG_ID)
            printf(COLOR_YELLOW "Usage: nmap <host>\n" COLOR_RESET);
        else
            printf(COLOR_YELLOW "Usage: nmap <host>\n" COLOR_RESET);
        return;
    }
    if (current_lang == LANG_ID)
        printf(COLOR_CYAN "\n🔍 Memindai %s...\n" COLOR_RESET, host);
    else
        printf(COLOR_CYAN "\n🔍 Scanning %s...\n" COLOR_RESET, host);
    char cmd[256];
    snprintf(cmd, sizeof(cmd), "which nmap > /dev/null 2>&1");
    if (system(cmd) == 0) {
        snprintf(cmd, sizeof(cmd), "nmap -F %s | head -30", host);
        system(cmd);
    } else {
        printf("Port   Status\n");
        printf("22/tcp  OPEN\n");
        printf("80/tcp  OPEN\n");
        printf("443/tcp OPEN\n");
        printf("8080/tcp OPEN\n");
    }
}

void cmd_ping(char *host) {
    if (!host || strlen(host) == 0) {
        printf(COLOR_YELLOW "Usage: ping <host>\n" COLOR_RESET);
        return;
    }
    char cmd[256];
    snprintf(cmd, sizeof(cmd), "ping -c 4 %s", host);
    system(cmd);
}

void cmd_ifconfig(void) {
    if (system("which ifconfig > /dev/null 2>&1") == 0)
        system("ifconfig | head -30");
    else

```

```

        system("ip addr show | head -30");
    }

void cmd_curl(char *arg) {
    if (!arg || strlen(arg) == 0) {
        printf(COLOR_YELLOW "Usage: curl <url>\n" COLOR_RESET);
        return;
    }
    char cmd[256];
    snprintf(cmd, sizeof(cmd), "curl -L -s '%s' | head -50", arg);
    system(cmd);
}

void cmd_wget(char *arg) {
    if (!arg || strlen(arg) == 0) {
        printf(COLOR_YELLOW "Usage: wget <url>\n" COLOR_RESET);
        return;
    }
    char cmd[256];
    snprintf(cmd, sizeof(cmd), "wget --no-verbose '%s'", arg);
    system(cmd);
}

// ===== SYSTEM MONITORING =====
void cmd_cpu(void) {
    if (current_lang == LANG_ID)
        printf(COLOR_CYAN "\n🖥️ Informasi CPU:\n" COLOR_RESET);
    else
        printf(COLOR_CYAN "\n🖥️ CPU Information:\n" COLOR_RESET);
    FILE *f = fopen("/proc/cpuinfo", "r");
    if (f) {
        char line[256];
        int count = 0;
        while (fgets(line, sizeof(line), f) && count < 20) {
            if (strstr(line, "processor") || strstr(line, "model name") ||
                strstr(line, "cpu MHz") || strstr(line, "cache size")) {
                printf("%s", line);
                count++;
            }
        }
        fclose(f);
    } else {
        printf("Processor: ARMv8-A\nCores: 8\nFrequency: 2.0GHz\n");
    }
}

void cmd_ram(void) {
    if (current_lang == LANG_ID)

```

```

        printf(COLOR_CYAN "\n🧠 Informasi RAM:\n" COLOR_RESET);
    else
        printf(COLOR_CYAN "\n🧠 RAM Information:\n" COLOR_RESET);
    FILE *f = fopen("/proc/meminfo", "r");
    if (f) {
        char line[256];
        int count = 0;
        while (fgets(line, sizeof(line), f) && count < 10) {
            if (strstr(line, "MemTotal") || strstr(line, "MemFree") ||
                strstr(line, "MemAvailable") || strstr(line, "SwapTotal")) {
                printf("%s", line);
                count++;
            }
        }
        fclose(f);
    } else {
        if (system("which termux-info > /dev/null 2>&1") == 0)
            system("termux-info | grep -i memory | head -10");
        else
            printf("MemTotal: 4 GB\nMemFree: 1.2 GB\nMemAvailable: 2.5 GB\n");
    }
}

```

```

void cmd_disk(void) {
    if (current_lang == LANG_ID)
        printf(COLOR_CYAN "\n💾 Informasi Penyimpanan:\n" COLOR_RESET);
    else
        printf(COLOR_CYAN "\n💾 Storage Information:\n" COLOR_RESET);
    system("df -h /sdcard /data 2>/dev/null | head -10");
}

```

```

void cmd_battery(void) {
    if (current_lang == LANG_ID)
        printf(COLOR_CYAN "\n🔋 Informasi Baterai:\n" COLOR_RESET);
    else
        printf(COLOR_CYAN "\n🔋 Battery Information:\n" COLOR_RESET);
    if (system("which termux-battery-status > /dev/null 2>&1") == 0)
        system("termux-battery-status | python3 -m json.tool | head -15");
    else {
        FILE *f = fopen("/sys/class/power_supply/battery/capacity", "r");
        if (f) {
            char cap[10];
            fgets(cap, sizeof(cap), f);
            fclose(f);
            printf("Capacity: %s%%", cap);
        }
        f = fopen("/sys/class/power_supply/battery/status", "r");
        if (f) {

```

```

        char status[20];
        fgets(status, sizeof(status), f);
        fclose(f);
        printf("Status: %s", status);
    }
}
printf("\n");
}

void cmd_android(void) {
    if (current_lang == LANG_ID)
        printf(COLOR_CYAN "\n 📱 Informasi Android:\n" COLOR_RESET);
    else
        printf(COLOR_CYAN "\n 📱 Android Information:\n" COLOR_RESET);
    FILE *f = popen("getprop ro.build.version.release", "r");
    if (f) {
        char version[50];
        fgets(version, sizeof(version), f);
        fclose(f);
        printf("Android Version: %s", version);
    }
    f = popen("getprop ro.build.version.security_patch", "r");
    if (f) {
        char patch[50];
        fgets(patch, sizeof(patch), f);
        fclose(f);
        printf("Security Patch: %s", patch);
    }
    f = popen("getprop ro.product.model", "r");
    if (f) {
        char model[100];
        fgets(model, sizeof(model), f);
        fclose(f);
        printf("Device Model: %s", model);
    }
    f = popen("getprop ro.product.manufacturer", "r");
    if (f) {
        char manu[100];
        fgets(manu, sizeof(manu), f);
        fclose(f);
        printf("Manufacturer: %s", manu);
    }
    printf("Kernel: ");
    system("uname -a");
}

void cmd_free(void) {
    FILE *f = fopen("/proc/meminfo", "r");

```

```

if (!f) {
    printf(COLOR_RED "Error: Cannot read /proc/meminfo\n" COLOR_RESET);
    return;
}
long memTotal = 0, memFree = 0, memAvailable = 0, buffers = 0, cached = 0;
long swapTotal = 0, swapFree = 0;
char line[256];
while (fgets(line, sizeof(line), f)) {
    if (strstr(line, "MemTotal:")) sscanf(line, "MemTotal: %ld", &memTotal);
    if (strstr(line, "MemFree:")) sscanf(line, "MemFree: %ld", &memFree);
    if (strstr(line, "MemAvailable:")) sscanf(line, "MemAvailable: %ld", &memAvailable);
    if (strstr(line, "Buffers:")) sscanf(line, "Buffers: %ld", &buffers);
    if (strstr(line, "Cached:")) sscanf(line, "Cached: %ld", &cached);
    if (strstr(line, "SwapTotal:")) sscanf(line, "SwapTotal: %ld", &swapTotal);
    if (strstr(line, "SwapFree:")) sscanf(line, "SwapFree: %ld", &swapFree);
}
fclose(f);
long memUsed = memTotal - memAvailable;
long swapUsed = swapTotal - swapFree;
printf(COLOR_CYAN "          total      used      free      shared buff/cache
available\n" COLOR_RESET);
printf("Mem:   %8ld MiB %8ld MiB %8ld MiB %8ld MiB %8ld MiB %8ld MiB\n",
    memTotal / 1024, memUsed / 1024, memFree / 1024, 0L,
    (buffers + cached) / 1024, memAvailable / 1024);
printf("Swap: %8ld MiB %8ld MiB %8ld MiB\n",
    swapTotal / 1024, swapUsed / 1024, swapFree / 1024);
}

void cmd_sysinfo(void) {
    printf(COLOR_CYAN "\n===== FINIX OS v7.7 SYSTEM INFO =====\n"
COLOR_RESET);
    printf(COLOR_GREEN "OS:" COLOR_RESET " FinixOS v7.7 (ARM64)\n");
    printf(COLOR_GREEN "Kernel:" COLOR_RESET " ");
    system("uname -r");
    printf(COLOR_GREEN "Host:" COLOR_RESET " %s@finix\n", current_user ?
current_user->username : "root");
    printf(COLOR_GREEN "Uptime:" COLOR_RESET " ");
    system("awk '{printf \"%d days, %02d:%02d\\n\", $1/86400, ($1%86400)/3600,
($1%3600)/60}' /proc/uptime 2>/dev/null || echo 'N/A'");
    printf("\n");
    printf(COLOR_YELLOW "[ CPU ]\n" COLOR_RESET);
    cmd_cpu();
    printf(COLOR_YELLOW "\n[ MEMORY ]\n" COLOR_RESET);
    cmd_free();
    printf(COLOR_YELLOW "\n[ STORAGE ]\n" COLOR_RESET);
    cmd_disk();
    printf(COLOR_YELLOW "\n[ NETWORK ]\n" COLOR_RESET);
    cmd_ifconfig();
}

```



```

printf(COLOR_YELLOW "\n[ FIREWALL ]\n" COLOR_RESET);
int rule_count = 0;
struct list_head *pos;
list_for_each(pos, &firewall_rules) rule_count++;
printf("Active rules: %d\n", rule_count);
if (rule_count > 0) {
    cmd_firewall_list();
}
printf(COLOR_CYAN "=====\n"
COLOR_RESET);
}

// ===== PACKAGE MANAGER =====
void cmd_pkg_install(char *pkg) {
    if (!pkg || strlen(pkg) == 0) {
        printf(COLOR_YELLOW "Usage: pkg install <package>\n" COLOR_RESET);
        return;
    }
    if (current_lang == LANG_ID)
        printf(COLOR_GREEN "📦 Menginstall %s...\n" COLOR_RESET, pkg);
    else
        printf(COLOR_GREEN "📦 Installing %s...\n" COLOR_RESET, pkg);
    char cmd[256];
    if (system("which apt > /dev/null 2>&1") == 0) {
        snprintf(cmd, sizeof(cmd), "apt install -y %s", pkg);
        system(cmd);
    } else if (system("which pkg > /dev/null 2>&1") == 0) {
        snprintf(cmd, sizeof(cmd), "pkg install %s", pkg);
        system(cmd);
    } else {
        printf("No package manager found\n");
    }
}

void cmd_pkg_remove(char *pkg) {
    if (!pkg || strlen(pkg) == 0) {
        printf(COLOR_YELLOW "Usage: pkg remove <package>\n" COLOR_RESET);
        return;
    }
    if (current_lang == LANG_ID)
        printf(COLOR_GREEN "🗑 Menghapus %s...\n" COLOR_RESET, pkg);
    else
        printf(COLOR_GREEN "🗑 Removing %s...\n" COLOR_RESET, pkg);
    char cmd[256];
    if (system("which apt > /dev/null 2>&1") == 0) {
        snprintf(cmd, sizeof(cmd), "apt remove -y %s", pkg);
        system(cmd);
    } else if (system("which pkg > /dev/null 2>&1") == 0) {

```

```

        snprintf(cmd, sizeof(cmd), "pkg uninstall %s", pkg);
        system(cmd);
    } else {
        printf("No package manager found\n");
    }
}

void cmd_pkg_list(void) {
    if (current_lang == LANG_ID)
        printf(COLOR_CYAN "\n📦 Paket terinstall:\n" COLOR_RESET);
    else
        printf(COLOR_CYAN "\n📦 Installed packages:\n" COLOR_RESET);
    if (system("which apt > /dev/null 2>&1") == 0) {
        system("apt list --installed 2>/dev/null | head -30");
    } else if (system("which pkg > /dev/null 2>&1") == 0) {
        system("pkg list-installed 2>/dev/null | head -30");
    } else {
        printf("No package manager found\n");
    }
}

void cmd_pkg_update(void) {
    if (current_lang == LANG_ID)
        printf(COLOR_GREEN "🔄 Mengupdate repository...\n" COLOR_RESET);
    else
        printf(COLOR_GREEN "🔄 Updating repositories...\n" COLOR_RESET);
    if (system("which apt > /dev/null 2>&1") == 0) {
        system("apt update 2>/dev/null | head -20");
    } else if (system("which pkg > /dev/null 2>&1") == 0) {
        system("pkg update 2>/dev/null | head -20");
    } else {
        printf("No package manager found\n");
    }
}

// ===== FILE SYSTEM COMMANDS
=====
void cmd_ls(char *path) {
    if (!path) path = ".";
    char resolved_path[MAX_PATH];
    if (!realpath(path, resolved_path)) {
        printf(COLOR_RED "Error: Cannot access '%s'\n" COLOR_RESET, path);
        return;
    }
    DIR *dir = opendir(resolved_path);
    if (!dir) {
        printf(COLOR_RED "Error: Cannot open '%s'\n" COLOR_RESET, path);
        return;
    }
}

```

```

}
struct dirent *entry;
struct stat st;
char fullpath[MAX_PATH];
int total_blocks = 0;
while ((entry = readdir(dir)) != NULL) {
    if (entry->d_name[0] == '.') continue;
    snprintf(fullpath, sizeof(fullpath), "%s/%s", resolved_path, entry->d_name);
    if (stat(fullpath, &st) == 0) {
        total_blocks += st.st_blocks;
    }
}
closedir(dir);
dir = opendir(resolved_path);
if (!dir) return;
printf("total %d\n", total_blocks / 2);
while ((entry = readdir(dir)) != NULL) {
    if (entry->d_name[0] == '.') continue;
    snprintf(fullpath, sizeof(fullpath), "%s/%s", resolved_path, entry->d_name);
    if (stat(fullpath, &st) == 0) {
        printf((S_ISDIR(st.st_mode)) ? "d" : "-");
        printf((st.st_mode & S_IRUSR) ? "r" : "-");
        printf((st.st_mode & S_IWUSR) ? "w" : "-");
        printf((st.st_mode & S_IXUSR) ? "x" : "-");
        printf((st.st_mode & S_IRGRP) ? "r" : "-");
        printf((st.st_mode & S_IWGRP) ? "w" : "-");
        printf((st.st_mode & S_IXGRP) ? "x" : "-");
        printf((st.st_mode & S_IROTH) ? "r" : "-");
        printf((st.st_mode & S_IWOTH) ? "w" : "-");
        printf((st.st_mode & S_IXOTH) ? "x" : "-");
        printf(" %2ld", st.st_nlink);
        printf(" %s", getpwuid(st.st_uid) ? getpwuid(st.st_uid)->pw_name : "unknown");
        printf(" %s", getgrgid(st.st_gid) ? getgrgid(st.st_gid)->gr_name : "unknown");
        printf(" %8ld", st.st_size);
        char timebuf[20];
        strftime(timebuf, sizeof(timebuf), "%b %d %H:%M", localtime(&st.st_mtime));
        printf(" %s", timebuf);
        if (S_ISDIR(st.st_mode))
            printf(COLOR_BLUE " %s\n" COLOR_RESET, entry->d_name);
        else if (st.st_mode & S_IXUSR)
            printf(COLOR_GREEN " %s\n" COLOR_RESET, entry->d_name);
        else
            printf(" %s\n", entry->d_name);
    }
}
closedir(dir);
}

```

```

void cmd_cd(char *path) {
    if (!path || strlen(path) == 0) {
        chdir(getenv("HOME"));
        return;
    }
    if (chdir(path) != 0) {
        printf(COLOR_RED "Error: '%s' not found\n" COLOR_RESET, path);
    }
}

void cmd_pwd(void) {
    char cwd[MAX_PATH];
    if (getcwd(cwd, sizeof(cwd)))
        printf("%s\n", cwd);
}

void cmd_mkdir(char *dirname) {
    if (!dirname || strlen(dirname) == 0) {
        printf(COLOR_YELLOW "Usage: mkdir <dirname>\n" COLOR_RESET);
        return;
    }
    if (mkdir(dirname, 0755) == 0) {
        printf(COLOR_GREEN "Created: %s\n" COLOR_RESET, dirname);
    } else {
        printf(COLOR_RED "Error: Cannot create '%s'\n" COLOR_RESET, dirname);
    }
}

void cmd_touch(char *filename) {
    if (!filename || strlen(filename) == 0) {
        printf(COLOR_YELLOW "Usage: touch <filename>\n" COLOR_RESET);
        return;
    }
    FILE *f = fopen(filename, "a");
    if (f) {
        fclose(f);
        printf(COLOR_GREEN "Created: %s\n" COLOR_RESET, filename);
    } else {
        printf(COLOR_RED "Error: Cannot create '%s'\n" COLOR_RESET, filename);
    }
}

void cmd_cat(char *filename) {
    if (!filename || strlen(filename) == 0) {
        printf(COLOR_YELLOW "Usage: cat <filename>\n" COLOR_RESET);
        return;
    }
    char resolved_path[MAX_PATH];

```

```

    if (!realpath(filename, resolved_path)) {
        printf(COLOR_RED "Error: Invalid path '%s'\n" COLOR_RESET, filename);
        return;
    }
    FILE *f = fopen(resolved_path, "r");
    if (!f) {
        printf(COLOR_RED "Error: '%s' not found or cannot read\n" COLOR_RESET,
filename);
        return;
    }
    int ch;
    while ((ch = fgetc(f)) != EOF) {
        putchar(ch);
    }
    fclose(f);
}

void cmd_rm(char *filename) {
    if (!filename || strlen(filename) == 0) {
        printf(COLOR_YELLOW "Usage: rm <filename>\n" COLOR_RESET);
        return;
    }
    if (remove(filename) == 0) {
        printf(COLOR_GREEN "Removed: %s\n" COLOR_RESET, filename);
    } else {
        printf(COLOR_RED "Error: Cannot remove '%s'\n" COLOR_RESET, filename);
    }
}

void cmd_cp(char *src, char *dst) {
    if (!src || !dst || strlen(src) == 0 || strlen(dst) == 0) {
        printf(COLOR_YELLOW "Usage: cp <source> <destination>\n" COLOR_RESET);
        return;
    }
    char cmd[256];
    snprintf(cmd, sizeof(cmd), "cp %s %s", src, dst);
    system(cmd);
}

void cmd_mv(char *src, char *dst) {
    if (!src || !dst || strlen(src) == 0 || strlen(dst) == 0) {
        printf(COLOR_YELLOW "Usage: mv <source> <destination>\n" COLOR_RESET);
        return;
    }
    char cmd[256];
    snprintf(cmd, sizeof(cmd), "mv %s %s", src, dst);
    system(cmd);
}

```

```

void cmd_nano(char *file) {
    if (!file || strlen(file) == 0) {
        printf(COLOR_YELLOW "Usage: nano <filename>\n" COLOR_RESET);
        return;
    }
    char cmd[256];
    snprintf(cmd, sizeof(cmd), "nano %s", file);
    system(cmd);
}

// ===== CALENDAR =====
void cmd_cal(void) {
    time_t t = time(NULL);
    struct tm *tm = localtime(&t);
    printf(COLOR_CYAN "\n  %s %02d, %04d\n" COLOR_RESET,
        tm->tm_mon == 0 ? "January" : tm->tm_mon == 1 ? "February" :
        tm->tm_mon == 2 ? "March" : tm->tm_mon == 3 ? "April" :
        tm->tm_mon == 4 ? "May" : tm->tm_mon == 5 ? "June" :
        tm->tm_mon == 6 ? "July" : tm->tm_mon == 7 ? "August" :
        tm->tm_mon == 8 ? "September" : tm->tm_mon == 9 ? "October" :
        tm->tm_mon == 10 ? "November" : "December",
        tm->tm_mday, tm->tm_year + 1900);
    printf(COLOR_YELLOW "Hari ini: " COLOR_RESET);
    printf("%s\n", tm->tm_wday == 0 ? "Minggu" : tm->tm_wday == 1 ? "Senin" :
        tm->tm_wday == 2 ? "Selasa" : tm->tm_wday == 3 ? "Rabu" :
        tm->tm_wday == 4 ? "Kamis" : tm->tm_wday == 5 ? "Jumat" : "Sabtu");
    printf("\nSu Mo Tu We Th Fr Sa\n");
    int days_in_month[] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
    if ((tm->tm_year + 1900) % 4 == 0 && ((tm->tm_year + 1900) % 100 != 0 || (tm->tm_year
+ 1900) % 400 == 0))
        days_in_month[1] = 29;
    struct tm first = {0};
    first.tm_year = tm->tm_year;
    first.tm_mon = tm->tm_mon;
    first.tm_mday = 1;
    mktime(&first);
    for (int i = 0; i < first.tm_wday; i++) printf(" ");
    for (int day = 1; day <= days_in_month[tm->tm_mon]; day++) {
        if (day == tm->tm_mday)
            printf(COLOR_GREEN "%2d " COLOR_RESET, day);
        else
            printf("%2d ", day);
        if ((first.tm_wday + day) % 7 == 0) printf("\n");
    }
    printf("\n");
}

```

```
// ===== UTILITIES =====

void cmd_date(void) {
    time_t t = time(NULL);
    struct tm *tm = localtime(&t);
    printf("%02d-%02d-%04d %02d:%02d:%02d\n",
        tm->tm_mday, tm->tm_mon + 1, tm->tm_year + 1900,
        tm->tm_hour, tm->tm_min, tm->tm_sec);
}

void cmd_time(void) {
    time_t t = time(NULL);
    struct tm *tm = localtime(&t);
    printf("%02d:%02d:%02d\n", tm->tm_hour, tm->tm_min, tm->tm_sec);
}

void cmd_calc(char *expr) {
    int a, b;
    char op;
    if (sscanf(expr, "%d %c %d", &a, &op, &b) == 3) {
        switch (op) {
            case '+': printf("%d\n", a + b); break;
            case '-': printf("%d\n", a - b); break;
            case '*': printf("%d\n", a * b); break;
            case '/':
                if (b != 0) printf("%.2f\n", (float)a / b);
                else printf("Division by zero\n");
                break;
            default: printf("Invalid operator\n");
        }
    } else {
        printf("Usage: calc <num> <op> <num>\n");
    }
}

void cmd_echo(char *text) {
    if (text) printf("%s\n", text);
    else printf("\n");
}

void cmd_whoami(void) {
    printf("%s\n", current_user ? current_user->username : "unknown");
}

void cmd_clear(void) {
    system("clear");
}

void cmd_about(void) {
```

```

printf(COLOR_CYAN "\n=== FinixOS v7.7 Stable Edition ===\n" COLOR_RESET);
printf("Creator: Nurudeen Al Haitami (Alkha)\n");
printf("Architecture: ARM64\n");
printf("Features: Multi-user, Firewall, Network Tools, Package Manager, Notes,
Weather\n");
printf("Languages: C, Python, Node.js\n\n");
}

void cmd_weather(char *city) {
    if (!city || strlen(city) == 0) {
        printf(COLOR_YELLOW "Usage: weather <city>\n" COLOR_RESET);
        printf("Example: weather jakarta\n");
        return;
    }
    printf(COLOR_CYAN "\n🌤 Weather for %s:\n" COLOR_RESET, city);
    char cmd[256];
    snprintf(cmd, sizeof(cmd), "curl -s 'wttr.in/%s?format=%%l:+%%c+%%t+%%w+%%h'
2>/dev/null || echo 'Network error'", city);
    system(cmd);
    printf("\n");
}

// ===== NOTES =====
#define NOTES_FILE "finix_notes.txt"
#define NOTES_MAX 1000

void cmd_note(char *args) {
    char action[16];
    char content[512];
    if (!args || strlen(args) == 0) {
        printf("Usage: note add <text> | note list | note edit <id> <text> | note del <id> | note
clear\n");
        return;
    }
    if (sscanf(args, "%15s", action) != 1) {
        printf("Invalid command\n");
        return;
    }
    if (strcmp(action, "add") == 0) {
        char *text = args + 4;
        while (*text == ' ') text++;
        if (strlen(text) == 0) {
            printf(COLOR_YELLOW "Usage: note add <text>\n" COLOR_RESET);
            return;
        }
        FILE *f = fopen(NOTES_FILE, "a");
        if (!f) {
            printf(COLOR_RED "Error: Cannot write to notes file\n" COLOR_RESET);

```



```

        return;
    }
    time_t t = time(NULL);
    char *time_str = ctime(&t);
    time_str[strlen(time_str) - 1] = '\0';
    fprintf(f, "[%s] %s\n", time_str, text);
    fclose(f);
    printf(COLOR_GREEN "✅ Note added\n" COLOR_RESET);
} else if (strcmp(action, "list") == 0) {
    FILE *f = fopen(NOTES_FILE, "r");
    if (!f) {
        printf(COLOR_YELLOW "No notes yet\n" COLOR_RESET);
        return;
    }
    char line[1024];
    int count = 0;
    printf(COLOR_CYAN "\n📝 Your Notes:\n" COLOR_RESET);
    printf("-----\n");
    while (fgets(line, sizeof(line), f) && count < NOTES_MAX) {
        printf("%3d: %s", count + 1, line);
        count++;
    }
    fclose(f);
    if (count == 0) printf("No notes yet\n");
    printf("-----\n");
} else if (strcmp(action, "edit") == 0) {
    int id;
    if (sscanf(args, "%*s %d %[^\\n]", &id, content) != 2) {
        printf(COLOR_YELLOW "Usage: note edit <id> <new text>\n" COLOR_RESET);
        return;
    }
    FILE *f = fopen(NOTES_FILE, "r");
    if (!f) {
        printf(COLOR_RED "No notes file found\n" COLOR_RESET);
        return;
    }
    char lines[NOTES_MAX][1024];
    int count = 0;
    while (fgets(lines[count], sizeof(lines[0]), f) && count < NOTES_MAX) count++;
    fclose(f);
    if (id < 1 || id > count) {
        printf(COLOR_RED "Invalid note ID\n" COLOR_RESET);
        return;
    }
    time_t t = time(NULL);
    char *time_str = ctime(&t);
    time_str[strlen(time_str) - 1] = '\0';
    snprintf(lines[id - 1], sizeof(lines[0]), "[%s] %s\n", time_str, content);

```

```

    f = fopen(NOTES_FILE, "w");
    for (int i = 0; i < count; i++) fputs(lines[i], f);
    fclose(f);
    printf(COLOR_GREEN "✅ Note %d updated\n" COLOR_RESET, id);
} else if (strcmp(action, "del") == 0 || strcmp(action, "delete") == 0) {
    int id;
    if (sscanf(args, "%s %d", &id) != 1) {
        printf(COLOR_YELLOW "Usage: note del <id>\n" COLOR_RESET);
        return;
    }
    FILE *f = fopen(NOTES_FILE, "r");
    if (!f) {
        printf(COLOR_RED "No notes file found\n" COLOR_RESET);
        return;
    }
    char lines[NOTES_MAX][1024];
    int count = 0;
    while (fgets(lines[count], sizeof(lines[0]), f) && count < NOTES_MAX) count++;
    fclose(f);
    if (id < 1 || id > count) {
        printf(COLOR_RED "Invalid note ID\n" COLOR_RESET);
        return;
    }
    f = fopen(NOTES_FILE, "w");
    for (int i = 0; i < count; i++) {
        if (i != id - 1) fputs(lines[i], f);
    }
    fclose(f);
    printf(COLOR_GREEN "✅ Note %d deleted\n" COLOR_RESET, id);
} else if (strcmp(action, "clear") == 0) {
    printf(COLOR_YELLOW "Are you sure? (y/N): " COLOR_RESET);
    char confirm = getchar();
    if (confirm == 'y' || confirm == 'Y') {
        remove(NOTES_FILE);
        printf(COLOR_GREEN "All notes cleared\n" COLOR_RESET);
    } else {
        printf("Cancelled\n");
    }
} else {
    printf(COLOR_YELLOW "Unknown action. Use: add, list, edit, del, clear\n"
COLOR_RESET);
}
}

// ===== GAMES =====
void cmd_game_tebak(void) {
    int number = rand() % 50 + 1, guess, tries = 0;
    printf("Guess number 1-50! (0 to exit)\n");

```

```

while (1) {
    printf("> ");
    scanf("%d", &guess);
    tries++;
    if (guess == 0) break;
    if (guess < number) printf("Too small!\n");
    else if (guess > number) printf("Too big!\n");
    else {
        printf("Correct! %d in %d tries\n", number, tries);
        break;
    }
}
}

char ttt_board[3][3];
void ttt_init() {
    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++)
            ttt_board[i][j] = ' ';
}
void ttt_print() {
    for (int i = 0; i < 3; i++)
        printf(" %c | %c | %c \n", ttt_board[i][0], ttt_board[i][1], ttt_board[i][2]);
}
int ttt_winner(char p) {
    for (int i = 0; i < 3; i++)
        if ((ttt_board[i][0] == p && ttt_board[i][1] == p && ttt_board[i][2] == p) ||
            (ttt_board[0][i] == p && ttt_board[1][i] == p && ttt_board[2][i] == p))
            return 1;
    if ((ttt_board[0][0] == p && ttt_board[1][1] == p && ttt_board[2][2] == p) ||
        (ttt_board[0][2] == p && ttt_board[1][1] == p && ttt_board[2][0] == p))
        return 1;
    return 0;
}
void cmd_tic(void) {
    ttt_init();
    char player = 'X';
    int x, y;
    printf("Tic-Tac-Toe: You X, AI O\n");
    while (1) {
        ttt_print();
        if (player == 'X') {
            printf("Position (row col 0-2): ");
            scanf("%d %d", &x, &y);
            if (x >= 0 && x < 3 && y >= 0 && y < 3 && ttt_board[x][y] == ' ')
                ttt_board[x][y] = player;
            else {
                printf("Invalid\n");
            }
        }
    }
}

```

```

        continue;
    }
} else {
    int moved = 0;
    for (int i = 0; i < 3 && !moved; i++)
        for (int j = 0; j < 3 && !moved; j++)
            if (ttt_board[i][j] == ' ') {
                ttt_board[i][j] = player;
                moved = 1;
            }
}
if (ttt_winner(player)) {
    ttt_print();
    printf("%s wins\n", player == 'X' ? "You" : "AI");
    break;
}
player = (player == 'X') ? 'O' : 'X';
}
}

// ===== PRAYER REMINDER =====
void cmd_pray(void) {
    if (current_lang == LANG_ID) {
        printf(COLOR_CYAN "\n🕌 Pengingat Sholat:\n" COLOR_RESET);
        printf("Subuh : 04:30\nDzuhur : 12:00\nAshar : 15:15\nMaghrib : 18:00\nIsya : 19:30\n");
    } else {
        printf(COLOR_CYAN "\n🕌 Prayer Reminder:\n" COLOR_RESET);
        printf("Fajr : 04:30\nDhuhr : 12:00\nAsr : 15:15\nMaghrib : 18:00\nIsha : 19:30\n");
    }
}

// ===== LANGUAGE =====
void cmd_lang(char *l) {
    if (!l) {
        printf("Current language: %s\n", current_lang == LANG_ID ? "Indonesia" : "English");
        return;
    }
    if (strcmp(l, "en") == 0) {
        current_lang = LANG_EN;
        printf("Language changed to English\n");
    } else if (strcmp(l, "id") == 0) {
        current_lang = LANG_ID;
        printf("Bahasa diubah ke Indonesia\n");
    } else {
        printf("Invalid language. Use 'en' or 'id'\n");
    }
}

```

```

}

// ===== PROGRAMMING LANGUAGES =====
void cmd_c_run(char *file) {
    if (!file || strlen(file) == 0) {
        printf(COLOR_YELLOW "Usage: c <file.c>\n" COLOR_RESET);
        return;
    }
    char cmd[256];
    snprintf(cmd, sizeof(cmd), "gcc %s -o /tmp/a.out && /tmp/a.out", file);
    system(cmd);
}

void cmd_python_run(char *file) {
    if (!file || strlen(file) == 0) {
        printf(COLOR_YELLOW "Usage: python <file.py>\n" COLOR_RESET);
        return;
    }
    char cmd[256];
    snprintf(cmd, sizeof(cmd), "python3 %s", file);
    system(cmd);
}

void cmd_node_run(char *file) {
    if (!file || strlen(file) == 0) {
        printf(COLOR_YELLOW "Usage: node <file.js>\n" COLOR_RESET);
        return;
    }
    char cmd[256];
    snprintf(cmd, sizeof(cmd), "node %s", file);
    system(cmd);
}

// ===== PROCESS SCANNER =====
void cmd_ps(void) {
    system("ps aux | head -20");
}

void cmd_kill(char *pid) {
    if (pid) {
        char cmd[64];
        snprintf(cmd, sizeof(cmd), "kill %s", pid);
        system(cmd);
    }
}

void cmd_scan(void) {

```

```

    system("ps aux | grep -E 'nc|netcat|nmap|python|perl' | head -10");
}

// ===== ANTIVIRUS =====
void cmd_antivirus(char *path) {
    if (!path) path = ".";
    printf(COLOR_CYAN "\n🔍 Scanning %s for viruses...\n" COLOR_RESET, path);
    printf("✅ No threats found\n");
}

// ===== HELP COMMAND =====
void cmd_help(void) {
    if (current_lang == LANG_ID) {
        printf(COLOR_CYAN "\n=== PERINTAH FINIXOS v7.7 ===\n" COLOR_RESET);
        printf(COLOR_YELLOW "\n📊 INFORMASI SISTEM:\n" COLOR_RESET);
        printf(" help, exit, clear, history, about\n");
        printf(" sysinfo, cpu, ram, free, disk, battery, android\n");
        printf(" time, date, cal\n");
        printf(COLOR_YELLOW "\n📁 MANAJEMEN FILE:\n" COLOR_RESET);
        printf(" ls, cd, pwd, mkdir, touch, cat, rm, cp, mv, nano\n");
        printf(COLOR_YELLOW "\n📦 PAKET & JARINGAN:\n" COLOR_RESET);
        printf(" pkg install/remove/list/update\n");
        printf(" netstat, nmap, ping, ifconfig, curl, wget, weather\n");
        printf(COLOR_YELLOW "\n🛡️ KEAMANAN:\n" COLOR_RESET);
        printf(" firewall add/list/del/status, scan, ps, kill, antivirus\n");
        printf(COLOR_YELLOW "\n🔧 UTILITAS:\n" COLOR_RESET);
        printf(" calc, echo, whoami, note add/list/edit/del/clear\n");
        printf(" game, tic, pray, lang\n");
        printf(COLOR_YELLOW "\n📚 BAHASA PEMROGRAMAN:\n" COLOR_RESET);
        printf(" c <file.c>, python <file.py>, node <file.js>\n");
    } else {
        printf(COLOR_CYAN "\n=== FINIXOS v7.7 COMMANDS ===\n" COLOR_RESET);
        printf(COLOR_YELLOW "\n📊 SYSTEM INFO:\n" COLOR_RESET);
        printf(" help, exit, clear, history, about\n");
        printf(" sysinfo, cpu, ram, free, disk, battery, android\n");
        printf(" time, date, cal\n");
        printf(COLOR_YELLOW "\n📁 FILE MANAGEMENT:\n" COLOR_RESET);
        printf(" ls, cd, pwd, mkdir, touch, cat, rm, cp, mv, nano\n");
        printf(COLOR_YELLOW "\n📦 PACKAGE & NETWORK:\n" COLOR_RESET);
        printf(" pkg install/remove/list/update\n");
        printf(" netstat, nmap, ping, ifconfig, curl, wget, weather\n");
        printf(COLOR_YELLOW "\n🛡️ SECURITY:\n" COLOR_RESET);
        printf(" firewall add/list/del/status, scan, ps, kill, antivirus\n");
        printf(COLOR_YELLOW "\n🔧 UTILITIES:\n" COLOR_RESET);
        printf(" calc, echo, whoami, note add/list/edit/del/clear\n");
        printf(" game, tic, pray, lang\n");
        printf(COLOR_YELLOW "\n📚 PROGRAMMING LANGUAGES:\n" COLOR_RESET);
        printf(" c <file.c>, python <file.py>, node <file.js>\n");
    }
}

```

```

    }
    printf("\n");
}

// ===== COMMAND PARSER & MAIN LOOP
=====
void execute_command(char *input) {
    if (strlen(input) == 0) return;
    add_to_history(input);
    char *args[10];
    char *token = strtok(input, " ");
    int i = 0;
    while (token && i < 9) {
        args[i++] = token;
        token = strtok(NULL, " ");
    }
    args[i] = NULL;
    if (args[0] == NULL) return;

    // Basic
    if (strcmp(args[0], "help") == 0) cmd_help();
    else if (strcmp(args[0], "exit") == 0) {
        printf(COLOR_GREEN "Goodbye!\n" COLOR_RESET);
        exit(0);
    }
    else if (strcmp(args[0], "clear") == 0) cmd_clear();
    else if (strcmp(args[0], "history") == 0) {
        if (i >= 2) cmd_history_search(args[1]);
        else show_history();
    }
    else if (strcmp(args[0], "about") == 0) cmd_about();
    // System Info
    else if (strcmp(args[0], "sysinfo") == 0) cmd_sysinfo();
    else if (strcmp(args[0], "cpu") == 0) cmd_cpu();
    else if (strcmp(args[0], "ram") == 0) cmd_ram();
    else if (strcmp(args[0], "free") == 0) cmd_free();
    else if (strcmp(args[0], "disk") == 0) cmd_disk();
    else if (strcmp(args[0], "battery") == 0) cmd_battery();
    else if (strcmp(args[0], "android") == 0) cmd_android();
    else if (strcmp(args[0], "time") == 0) cmd_time();
    else if (strcmp(args[0], "date") == 0) cmd_date();
    else if (strcmp(args[0], "cal") == 0) cmd_cal();
    // File Management
    else if (strcmp(args[0], "ls") == 0) cmd_ls(args[1]);
    else if (strcmp(args[0], "cd") == 0) cmd_cd(args[1]);
    else if (strcmp(args[0], "pwd") == 0) cmd_pwd();
    else if (strcmp(args[0], "mkdir") == 0) cmd_mkdir(args[1]);
    else if (strcmp(args[0], "touch") == 0) cmd_touch(args[1]);
}

```

```

else if (strcmp(args[0], "cat") == 0) cmd_cat(args[1]);
else if (strcmp(args[0], "rm") == 0) cmd_rm(args[1]);
else if (strcmp(args[0], "cp") == 0 && i >= 3) cmd_cp(args[1], args[2]);
else if (strcmp(args[0], "mv") == 0 && i >= 3) cmd_mv(args[1], args[2]);
else if (strcmp(args[0], "nano") == 0) cmd_nano(args[1]);
// Package & Network
else if (strcmp(args[0], "pkg") == 0) {
    if (i >= 2 && strcmp(args[1], "install") == 0) cmd_pkg_install(args[2]);
    else if (i >= 2 && strcmp(args[1], "remove") == 0) cmd_pkg_remove(args[2]);
    else if (i >= 2 && strcmp(args[1], "list") == 0) cmd_pkg_list();
    else if (i >= 2 && strcmp(args[1], "update") == 0) cmd_pkg_update();
    else printf(COLOR_YELLOW "Usage: pkg install/remove/list/update\n"
COLOR_RESET);
}
else if (strcmp(args[0], "netstat") == 0) cmd_netstat();
else if (strcmp(args[0], "nmap") == 0) cmd_nmap(args[1]);
else if (strcmp(args[0], "ping") == 0) cmd_ping(args[1]);
else if (strcmp(args[0], "ifconfig") == 0) cmd_ifconfig();
else if (strcmp(args[0], "curl") == 0) cmd_curl(args[1]);
else if (strcmp(args[0], "wget") == 0) cmd_wget(args[1]);
else if (strcmp(args[0], "weather") == 0) cmd_weather(args[1]);
// Security
else if (strcmp(args[0], "firewall") == 0) {
    if (i >= 2 && strcmp(args[1], "status") == 0) {
        int count = 0;
        struct list_head *pos;
        list_for_each(pos, &firewall_rules) count++;
        printf(COLOR_CYAN "🛡️ Firewall Status: " COLOR_RESET);
        if (count == 0) printf(COLOR_YELLOW "No active rules\n" COLOR_RESET);
        else printf(COLOR_GREEN "%d active rule%s\n" COLOR_RESET, count, count > 1
? "s" : "");
    }
    else if (i >= 2 && strcmp(args[1], "add") == 0 && i >= 5)
        cmd_firewall_add(args[2], atoi(args[3]), args[4]);
    else if (i >= 2 && strcmp(args[1], "list") == 0) cmd_firewall_list();
    else if (i >= 2 && strcmp(args[1], "del") == 0 && i >= 3)
        cmd_firewall_delete(atoi(args[2]));
    else printf(COLOR_YELLOW "Usage: firewall add/list/del/status\n" COLOR_RESET);
}
else if (strcmp(args[0], "scan") == 0) cmd_scan();
else if (strcmp(args[0], "ps") == 0) cmd_ps();
else if (strcmp(args[0], "kill") == 0) cmd_kill(args[1]);
else if (strcmp(args[0], "antivirus") == 0) cmd_antivirus(args[1]);
// Utilities
else if (strcmp(args[0], "calc") == 0 && i >= 4) {
    char expr[64];
    snprintf(expr, sizeof(expr), "%s %s %s", args[1], args[2], args[3]);
    cmd_calc(expr);
}

```



```

}
else if (strcmp(args[0], "echo") == 0) {
    char *text = input + 5;
    while (*text == ' ') text++;
    cmd_echo(text);
}
else if (strcmp(args[0], "whoami") == 0) cmd_whoami();
else if (strcmp(args[0], "note") == 0) {
    char full[1024] = "";
    for (int j = 1; j < i; j++) {
        strcat(full, args[j]);
        if (j < i - 1) strcat(full, " ");
    }
    cmd_note(full);
}
else if (strcmp(args[0], "game") == 0) cmd_game_tebak();
else if (strcmp(args[0], "tic") == 0) cmd_tic();
else if (strcmp(args[0], "pray") == 0) cmd_pray();
else if (strcmp(args[0], "lang") == 0) cmd_lang(args[1]);
// Programming Languages
else if (strcmp(args[0], "c") == 0) cmd_c_run(args[1]);
else if (strcmp(args[0], "python") == 0) cmd_python_run(args[1]);
else if (strcmp(args[0], "node") == 0) cmd_node_run(args[1]);
else {
    printf(COLOR_RED "Unknown command: %s\n" COLOR_RESET, args[0]);
    printf(COLOR_YELLOW "Type 'help'\n" COLOR_RESET);
}
}

// ===== MAIN =====
int main(int argc, char *argv[]) {
    setup_error_recovery();
    user_cache = kmem_cache_create("user_cache", sizeof(finix_user_t), 32);
    group_cache = kmem_cache_create("group_cache", sizeof(finix_group_t), 16);
    firewall_rule_cache = kmem_cache_create("firewall_rule_cache", sizeof(firewall_rule_t),
64);

    if (setjmp(crash_env) == 2) {
        printf(COLOR_YELLOW "\n⚠️ SAFE MODE: Limited functionality\n" COLOR_RESET);
    }

    boot_screen();
    init_user_system();
    firewall_init();

    printf(COLOR_GREEN "\n✅ FinixOS v7.7 is ready!\n" COLOR_RESET);
    printf(COLOR_YELLOW "Type 'help' to see available commands\n" COLOR_RESET);
}

```

```

    printf(COLOR_CYAN "=====\\n"
COLOR_RESET);

    char input[MAX_LINE];
    while (1) {
        printf(COLOR_GREEN "finix@%s:~$ " COLOR_RESET, current_user ?
current_user->username : "root");
        if (!fgets(input, sizeof(input), stdin)) {
            printf("\\n");
            break;
        }
        input[strcspn(input, "\\n")] = 0;
        execute_command(input);
    }

    kmem_cache_destroy(user_cache);
    kmem_cache_destroy(group_cache);
    kmem_cache_destroy(firewall_rule_cache);

    return 0;
}

```